```
public class IndexModel : PageModel
{
    private readonly RecipeService _service;
    private readonly ILogger<IndexModel> _log;                    ❶

    public ICollection<RecipeSummaryViewModel> Recipes { get; set; }

    public IndexModel(
        RecipeService service,
        ILogger<IndexModel> log)                                  ❶
    {
        _service = service;
        _log = log;                                               ❶
    }

    public void OnGet()
    {
        Recipes = _service.GetRecipes();
        _log.LogInformation(                                      ❷
            "Loaded {RecipeCount} recipes", Recipes.Count);       ❷
    }
}
```

❶ Injects the generic ILogger<T> using DI, which implements ILogger

❷ Writes an Information-level log. The RecipeCount variable is substituted in the message.

In this example you're using one of the many extension methods on `ILogger` to create the log message, `LogInformation()`. There are many extension methods on `ILogger` that let you easily specify a `LogLevel` for the message.

DEFINITION The *log level* of a log is how important it is and is defined by the `LogLevel` enum. Every log message has a log level.

You can also see that the message you pass to the `LogInformation` method has a placeholder indicated by braces, `{RecipeCount}`, and you pass an additional parameter, `Recipes .Count`, to the logger. The logger replaces the placeholder with the parameter at runtime. Placeholders